Homework 1 — Coding

Dataset: AG News (4 classes)

Undergrad Required:

- Part 1: Binary BoW
- Part 2: Count BoW
- Part 3: 2-layer neural classifier
- Part 4: Train/evaluate and compare Binary vs Count

Graduate Required / Undergrad Bonus:

- Part 5: TF-IDF BoW + same classifier + short analysis

Hints

- Tokenizer, vocabulary, and dataset split are provided.
- Use PyTorch autograd for backpropagation.
- Do **not** use transformers or LLM embeddings/models.

Complete the `# TODO` sections.

0. Setup

In [13]:
```
!pip install datasets --break-system-packages
```

```
Requirement already satisfied: datasets in /opt/homebrew/lib/python3.12/site
-packages (4.5.0)
Requirement already satisfied: filelock in /opt/homebrew/lib/python3.12/site
-packages (from datasets) (3.16.1)
Requirement already satisfied: numpy>=1.17 in /opt/homebrew/lib/python3.12/s
ite-packages (from datasets) (1.26.4)
Requirement already satisfied: pyarrow>=21.0.0 in /opt/homebrew/lib/python3.
12/site-packages (from datasets) (23.0.0)
Requirement already satisfied: dill<0.4.1,>=0.3.0 in /opt/homebrew/lib/pytho
n3.12/site-packages (from datasets) (0.4.0)
Requirement already satisfied: pandas in /opt/homebrew/lib/python3.12/site-p
ackages (from datasets) (2.2.3)
Requirement already satisfied: requests>=2.32.2 in /opt/homebrew/lib/python
3.12/site-packages (from datasets) (2.32.3)
Requirement already satisfied: httpx<1.0.0 in /opt/homebrew/lib/python3.12/s
ite-packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /opt/homebrew/lib/python3.12/
site-packages (from datasets) (4.67.1)
Requirement already satisfied: xxhash in /opt/homebrew/lib/python3.12/site-p
ackages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.19 in /opt/homebrew/lib/pyt
hon3.12/site-packages (from datasets) (0.70.18)
Requirement already satisfied: fsspec<=2025.10.0,>=2023.1.0 in /opt/homebre
w/lib/python3.12/site-packages (from fsspec[http]<=2025.10.0,>=2023.1.0->dat
asets) (2024.12.0)
Requirement already satisfied: huggingface-hub<2.0,>=0.25.0 in /opt/homebre
w/lib/python3.12/site-packages (from datasets) (0.27.0)
Requirement already satisfied: packaging in /opt/homebrew/lib/python3.12/sit
e-packages (from datasets) (24.2)
Requirement already satisfied: pyyaml>=5.1 in /opt/homebrew/lib/python3.12/s
ite-packages (from datasets) (6.0.2)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /opt/homebrew/l
ib/python3.12/site-packages (from fsspec[http]<=2025.10.0,>=2023.1.0->datase
ts) (3.13.3)
Requirement already satisfied: anyio in /opt/homebrew/lib/python3.12/site-pa
ckages (from httpx<1.0.0->datasets) (4.9.0)
Requirement already satisfied: certifi in /opt/homebrew/lib/python3.12/site-
packages (from httpx<1.0.0->datasets) (2024.12.14)
Requirement already satisfied: httpcore==1.* in /opt/homebrew/lib/python3.1
2/site-packages (from httpx<1.0.0->datasets) (1.0.9)
Requirement already satisfied: idna in /opt/homebrew/lib/python3.12/site-pac
kages (from httpx<1.0.0->datasets) (3.10)
Requirement already satisfied: h11>=0.16 in /opt/homebrew/lib/python3.12/sit
e-packages (from httpcore==1.*->httpx<1.0.0->datasets) (0.16.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /opt/homebrew/l
ib/python3.12/site-packages (from huggingface-hub<2.0,>=0.25.0->datasets)
(4.15.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /opt/homebrew/lib/
python3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=202
5.10.0,>=2023.1.0->datasets) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in /opt/homebrew/lib/python
3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.10.
0,>=2023.1.0->datasets) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /opt/homebrew/lib/python3.1
2/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.10.0,>=
2023.1.0->datasets) (25.4.0)
```

```
Requirement already satisfied: frozenlist>=1.1.1 in /opt/homebrew/lib/python
3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.10.
0,>=2023.1.0->datasets) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /opt/homebrew/lib/pyth
on3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.1
0.0,>=2023.1.0->datasets) (6.7.1)
Requirement already satisfied: propcache>=0.2.0 in /opt/homebrew/lib/python
3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.10.
0,>=2023.1.0->datasets) (0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /opt/homebrew/lib/python
3.12/site-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.10.
0,>=2023.1.0->datasets) (1.22.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /opt/homebrew/li
b/python3.12/site-packages (from requests>=2.32.2->datasets) (3.4.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/homebrew/lib/pytho
n3.12/site-packages (from requests>=2.32.2->datasets) (2.2.3)
Requirement already satisfied: sniffio>=1.1 in /opt/homebrew/lib/python3.12/
site-packages (from anyio->httpx<1.0.0->datasets) (1.3.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/homebrew/lib/p
ython3.12/site-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/homebrew/lib/python3.12/
site-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /opt/homebrew/lib/python3.1
2/site-packages (from pandas->datasets) (2024.2)
Requirement already satisfied: six>=1.5 in /opt/homebrew/lib/python3.12/site
-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
```

In [14]:

```python
# Local install (inside your conda env):
#   pip install datasets torch scikit-learn numpy

import math, random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from datasets import load_dataset
from sklearn.metrics import accuracy_score, f1_score
```

In [15]:

```python
SEED = 120
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

Out[15]:

```
device(type='cpu')
```

1. Load AG News + create train/dev split (provided)

In [16]:

```python
ds = load_dataset("ag_news")
train_ds = ds["train"]; test_ds = ds["test"]

dev_ratio = 0.1
```

```
n = len(train_ds)
idx = np.arange(n)
rng = np.random.default_rng(SEED)
rng.shuffle(idx)

n_dev = int(n * dev_ratio)
dev_idx = idx[:n_dev]; tr_idx = idx[n_dev:]

train_texts = [train_ds[int(i)]["text"] for i in tr_idx]
train_y     = np.array([train_ds[int(i)]["label"] for i in tr_idx], dtype=np

dev_texts = [train_ds[int(i)]["text"] for i in dev_idx]
dev_y     = np.array([train_ds[int(i)]["label"] for i in dev_idx], dtype=np.

test_texts = list(test_ds["text"])
test_y     = np.array(test_ds["label"], dtype=np.int64)

len(train_texts), len(dev_texts), len(test_texts)
```

Out[16]: (108000, 12000, 7600)

In [17]:
```
'''
This cell is used to speed up debugging of your training code by using only
development, and test data. Remember to set it to None for the final run bef
'''

LIMIT_TRAIN = 30000
LIMIT_DEV   = 5000
LIMIT_TEST  = 5000

if LIMIT_TRAIN is not None:
    train_texts = train_texts[:LIMIT_TRAIN]
    train_y = train_y[:LIMIT_TRAIN]
if LIMIT_DEV is not None:
    dev_texts = dev_texts[:LIMIT_DEV]
    dev_y = dev_y[:LIMIT_DEV]
if LIMIT_TEST is not None:
    test_texts = test_texts[:LIMIT_TEST]
    test_y = test_y[:LIMIT_TEST]

len(train_texts), len(dev_texts), len(test_texts)
```

Out[17]: (30000, 5000, 5000)

2. Tokenizer (provided)

In [18]:
```
import re
def tokenize(text: str):
    """Word-level tokenizer: lowercase + alphanumeric tokens."""
    return re.findall(r"[a-z0-9]+", text.lower())
```

3. Vocabulary (provided; built from TRAIN ONLY)

In [19]:
```
from collections import Counter
PAD_TOKEN = "<pad>"
UNK_TOKEN = "<unk>"

def build_vocab(texts, vocab_size=20000, min_df=2):
    df = Counter(); tf = Counter()
    for txt in texts:
        toks = tokenize(txt)
        tf.update(toks); df.update(set(toks))
    candidates = [(tf[t], t) for t in tf if df[t] >= min_df]
    candidates.sort(key=lambda x: (-x[0], x[1]))
    kept = [t for _, t in candidates[: max(0, vocab_size-2)]]
    vocab = {PAD_TOKEN:0, UNK_TOKEN:1}
    for t in kept:
        if t not in vocab: vocab[t] = len(vocab)
    return vocab

vocab = build_vocab(train_texts)
V = len(vocab)
print("Vocab size:", V)
```

```
Vocab size: 20000
```

Part 1 — Binary Bag-of-Words

In [21]:
```
def vectorize_bow_binary(texts, vocab, max_len=200):
    """Binary BoW: X[i,j]=1 if token j appears in doc i."""
    V = len(vocab)
    X = np.zeros((len(texts), V), dtype=np.float32)

    for i, text in enumerate(texts):
        tokens = tokenize(text)[:max_len]  # Truncate to max_len
        for token in tokens:
            if token in vocab:
                X[i, vocab[token]] = 1.0
            else:
                X[i, vocab[UNK_TOKEN]] = 1.0  # Map OOV to <unk>

    return X

Xtr_bin = vectorize_bow_binary(train_texts, vocab)
Xdv_bin = vectorize_bow_binary(dev_texts, vocab)
Xte_bin = vectorize_bow_binary(test_texts, vocab)
Xtr_bin.shape, Xdv_bin.shape, Xte_bin.shape
```

Out[21]:
```
((30000, 20000), (5000, 20000), (5000, 20000))
```

Part 2 — Count Bag-of-Words

In [23]:
```
def vectorize_bow_count(texts, vocab, max_len=200):
    """Count BoW: X[i,j]=number of occurrences of token j in doc i."""
    V = len(vocab)
    X = np.zeros((len(texts), V), dtype=np.float32)
```

```
    for i, text in enumerate(texts):
        tokens = tokenize(text)[:max_len]
        for token in tokens:
            if token in vocab:
                X[i, vocab[token]] += 1.0
            else:
                X[i, vocab[UNK_TOKEN]] += 1.0

    return X

Xtr_cnt = vectorize_bow_count(train_texts, vocab)
Xdv_cnt = vectorize_bow_count(dev_texts, vocab)
Xte_cnt = vectorize_bow_count(test_texts, vocab)
Xtr_cnt.shape, Xdv_cnt.shape, Xte_cnt.shape
```

Out[23]: ((30000, 20000), (5000, 20000), (5000, 20000))

Part 3 — 2-layer Neural Network Classifier

In [24]:
```
class TwoLayerNN(nn.Module):
    """
    Implement a 2-layer neural network classifier:
      Linear(input_dim -> hidden_dim) -> ReLU -> Linear(hidden_dim -> num_cl
    The output should be logits of shape (B, 4).
    """
    def __init__(self, input_dim, hidden_dim=256, num_classes=4):
        super().__init__()
        # Define the two linear layers and activation
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        # Forward pass: x -> Linear -> ReLU -> Linear -> logits
        h = self.fc1(x)
        h = self.relu(h)
        logits = self.fc2(h)
        return logits
```

Part 4 — Training loop

In [ ]:
```
def train_model(Xtr, ytr, Xdv, ydv, epochs=5, lr=1e-3):
    """Train on full batches (no mini-batching) for simplicity."""
    model = TwoLayerNN(input_dim=Xtr.shape[1]).to(device)

    # Use Adam optimizer and CrossEntropy loss
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()

    # Convert datasets to torch tensors
    Xtr_t = torch.tensor(Xtr, dtype=torch.float32).to(device)
    ytr_t = torch.tensor(ytr, dtype=torch.long).to(device)
```

```
    Xdv_t = torch.tensor(Xdv, dtype=torch.float32).to(device)
    ydv_t = torch.tensor(ydv, dtype=torch.long).to(device)

    for ep in range(1, epochs+1):
        model.train()

        # 1. Clear old gradients
        optimizer.zero_grad()

        # 2. Forward pass
        logits = model(Xtr_t)

        # 3. Compute training loss
        loss = criterion(logits, ytr_t)

        # 4. Backward pass (autograd)
        loss.backward()

        # 5. Parameter update
        optimizer.step()

        # Evaluation on dev set
        model.eval()
        with torch.no_grad():
            pred = model(Xdv_t).argmax(dim=-1).cpu().numpy()

        acc = accuracy_score(ydv, pred)
        mf1 = f1_score(ydv, pred, average="macro")
        print(f"Epoch {ep} | training loss={loss.item():.4f} | dev acc={acc:

    return model
```

Part 4 — Train on Binary vs Count BoW and compare metrics (provided)

In [26]:

```
def eval_model(model, X, y):
    model.eval()
    X_t = torch.tensor(X, dtype=torch.float32).to(device)
    with torch.no_grad():
        pred = model(X_t).argmax(dim=-1).cpu().numpy()
    return {
        "accuracy": float(accuracy_score(y, pred)),
        "macro_f1": float(f1_score(y, pred, average="macro")),
    }

print("Training on BINARY BoW...")
model_bin = train_model(Xtr_bin, train_y, Xdv_bin, dev_y, epochs=5, lr=1e-3)
dev_bin = eval_model(model_bin, Xdv_bin, dev_y)
test_bin = eval_model(model_bin, Xte_bin, test_y)

print("Training on COUNT BoW...")
model_cnt = train_model(Xtr_cnt, train_y, Xdv_cnt, dev_y, epochs=5, lr=1e-3)
dev_cnt = eval_model(model_cnt, Xdv_cnt, dev_y)
test_cnt = eval_model(model_cnt, Xte_cnt, test_y)
```

```
print("Binary dev/test:", dev_bin, test_bin)
print("Count  dev/test:", dev_cnt, test_cnt)
```

```
Training on BINARY BoW...
Epoch 1 | training loss=1.3869 | dev acc=0.5294 | dev macro-f1=0.4962
Epoch 2 | training loss=1.3488 | dev acc=0.7882 | dev macro-f1=0.7833
Epoch 3 | training loss=1.2968 | dev acc=0.8420 | dev macro-f1=0.8401
Epoch 4 | training loss=1.2332 | dev acc=0.8566 | dev macro-f1=0.8553
Epoch 5 | training loss=1.1669 | dev acc=0.8620 | dev macro-f1=0.8608
Training on COUNT BoW...
Epoch 1 | training loss=1.3872 | dev acc=0.6640 | dev macro-f1=0.6642
Epoch 2 | training loss=1.3388 | dev acc=0.8430 | dev macro-f1=0.8420
Epoch 3 | training loss=1.2747 | dev acc=0.8698 | dev macro-f1=0.8700
Epoch 4 | training loss=1.1970 | dev acc=0.8740 | dev macro-f1=0.8746
Epoch 5 | training loss=1.1168 | dev acc=0.8780 | dev macro-f1=0.8789
Binary dev/test: {'accuracy': 0.862, 'macro_f1': 0.8608441357810598} {'accur
acy': 0.8622, 'macro_f1': 0.8596461114353106}
Count  dev/test: {'accuracy': 0.878, 'macro_f1': 0.8788595951231623} {'accur
acy': 0.8748, 'macro_f1': 0.8730707770464805}
```

Required comparison (write your answer here)

1. Which is better on dev/test: **Binary BoW** or **Count BoW**?
2. Give 1--2 reasons (e.g., repeated topical words, length effects, noise).
3. If the difference is small, why might both be strong on AG News?

1. Which is better on dev/test: **Count BoW** is better, outperforming Binary BoW by ~1.6% on the development set and ~1.26% on the test set.
2. Give 1--2 reasons:
   - **Signal Intensity:** Count BoW captures the frequency of topical keywords. In news snippets, a word like "market" or "win" appearing multiple times provides a stronger, more confident signal to the classifier than a single mention.
   - **Feature Richness:** Binary BoW discards magnitude information, treating all word occurrences as equal, whereas Count BoW allows the model to learn that higher word density correlates more strongly with specific categories.
3. If the difference is small, why might both be strong on AG News?
   - **Short Document Length:** Since AG News contains short headlines and snippets, most informative words naturally only appear once or twice. In these cases, the sparse vectors for Binary and Count representations are nearly identical, leading to similar performance.

Part 5 (Graduate Required / Undergrad Bonus) — TF-IDF BoW (TODO)

In [27]:
```
def compute_idf(train_texts, vocab, max_len=200):
    """
```

```
    Compute IDF over TRAIN ONLY.
    df(token) = number of training docs containing token at least once
    idf(token) = log(N / df(token))
    """
    N = len(train_texts)
    V = len(vocab)
    df = np.zeros(V, dtype=np.float32)

    for text in train_texts:
        tokens = set(tokenize(text)[:max_len])
        for token in tokens:
            if token in vocab:
                df[vocab[token]] += 1
            else:
                # We can track df for <unk> too
                df[vocab[UNK_TOKEN]] += 1

    # Use max(1, df) to avoid division by zero, though min_df=2 in build_voc
    idf = np.log(N / np.maximum(1, df))
    return idf.astype(np.float32)

def vectorize_tfidf(texts, vocab, idf, max_len=200):
    """TF-IDF with raw TF: X[i,j] = tf(i,j) * idf(j)."""
    # First get the raw counts (TF)
    X = vectorize_bow_count(texts, vocab, max_len)

    # Multiply each row by the IDF vector
    # X is (N, V) and idf is (V,)
    X_tfidf = X * idf

    return X_tfidf

idf = compute_idf(train_texts, vocab)
Xtr_tfidf = vectorize_tfidf(train_texts, vocab, idf)
Xdv_tfidf = vectorize_tfidf(dev_texts, vocab, idf)
Xte_tfidf = vectorize_tfidf(test_texts, vocab, idf)

print("Training on TF-IDF BoW...")
model_tfidf = train_model(Xtr_tfidf, train_y, Xdv_tfidf, dev_y, epochs=5, lr
dev_tfidf = eval_model(model_tfidf, Xdv_tfidf, dev_y)
test_tfidf = eval_model(model_tfidf, Xte_tfidf, test_y)

print("TF-IDF dev/test:", dev_tfidf, test_tfidf)
```

```
Training on TF-IDF BoW...
Epoch 1 | training loss=1.3886 | dev acc=0.8656 | dev macro-f1=0.8664
Epoch 2 | training loss=1.1463 | dev acc=0.8874 | dev macro-f1=0.8880
Epoch 3 | training loss=0.9012 | dev acc=0.8938 | dev macro-f1=0.8945
Epoch 4 | training loss=0.6703 | dev acc=0.8972 | dev macro-f1=0.8980
Epoch 5 | training loss=0.4931 | dev acc=0.8992 | dev macro-f1=0.9000
TF-IDF dev/test: {'accuracy': 0.8992, 'macro_f1': 0.9000032485095907} {'accu
racy': 0.8896, 'macro_f1': 0.8884607478816314}
```

Graduate/Bonus analysis (write your answer here)

Compare **TF-IDF** against Binary/Count BoW:

- Is TF-IDF better or worse on AG News?
- Why might TF-IDF help topic classification? When might it hurt?

- **Is TF-IDF better or worse on AG News?** TF-IDF is significantly **better**. It reached 89.92% dev accuracy, which is 2.1% higher than Count BoW and 3.7% higher than Binary BoW. It also converges much faster, outperforming Binary's final results in its very first epoch.
- **Why might TF-IDF help topic classification? When might it hurt?**
  - **Help:** It effectively filters "noise" by penalizing high-frequency words (e.g., "said", "from") that appear across all categories, while amplifying rare, discriminative words (e.g., "semiconductor", "league") that are highly specific to a single topic.
  - **Hurt:** It can hurt if there is a **domain shift**. Since it relies heavily on the specific rare words of the training set, it may be less robust than Binary BoW if the test set introduces new terminology or if the importance of rare words changes significantly between datasets.